

TP N°2 : Manipulation des capteurs et actionneurs avec Arduino

I. Objectifs pédagogiques

- Comprendre le **principe physique et électronique** de trois capteurs analogiques et numériques.
- Apprendre à **câbler, mesurer et interpréter** les signaux des capteurs.
- Programmer l'Arduino pour acquérir et afficher les mesures sur le **Moniteur Série**.
- Simuler le montage sur **Tinkercad Circuits**.

II. Exercice 1 : Capteur de distance

Le capteur ultrasonique mesure la **distance** en envoyant une onde sonore et en mesurant le **temps de retour de l'écho** :

$$Distance = \frac{(v_{son} \times durée)}{2}$$

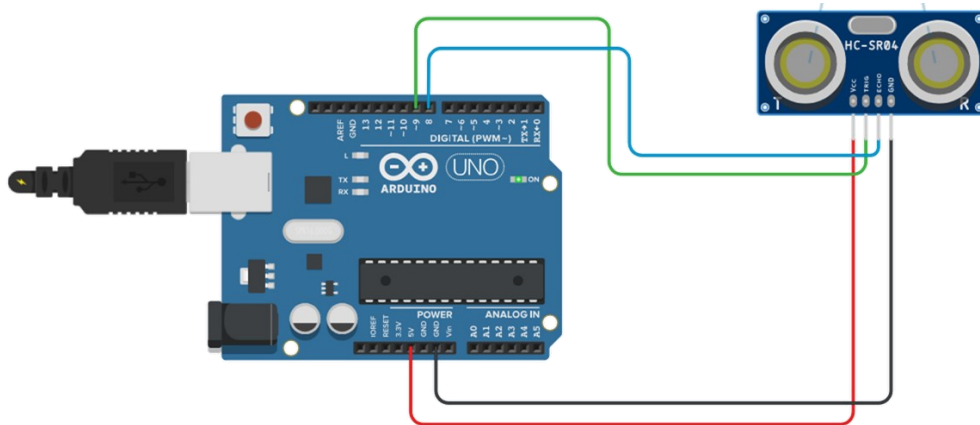
où $v_{son} = 340$ m/s.



- **VCC** : Connectez-la à une source d'alimentation de 5V.
- **GND** : Connectez-la à la masse (ground) de votre circuit.
- **Trig** (Trigger) : Cette broche est utilisée pour déclencher l'émission de l'onde ultrasonique.
 - Elle doit être connectée à une broche de sortie d'un microcontrôleur (comme Arduino).
 - Pour lancer une mesure, envoyez une impulsion haute (état haut) d'au moins $10\mu s$ sur cette broche.
- **Echo** : Cette broche fournit le signal de retour.
 - Elle doit être connectée à une broche d'entrée d'un microcontrôleur.
 - Lorsque le signal ultrasonique revient, cette broche passe à l'état haut pendant une durée proportionnelle à la distance. La mesure de cette durée permet de calculer la distance.

- Câblage

Broche HC-SR04	Arduino UNO
VCC	5V
GND	GND
TRIG	D9
ECHO	D8

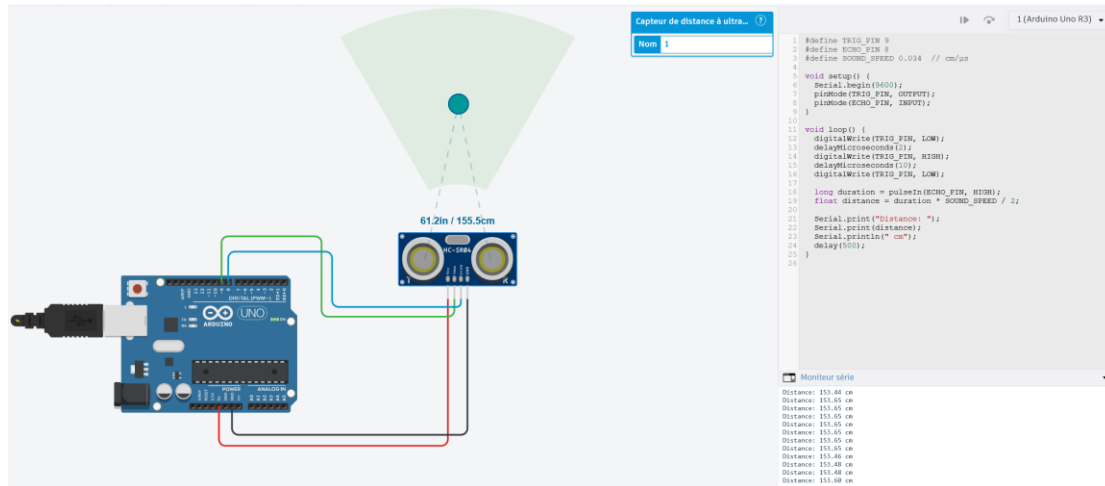


- Code Arduino

```
1  #define TRIG_PIN 9
2  #define ECHO_PIN 8
3  #define SOUND_SPEED 0.034 // cm/μs
4
5  void setup() {
6      Serial.begin(9600);
7      pinMode(TRIG_PIN, OUTPUT);
8      pinMode(ECHO_PIN, INPUT);
9  }
10
11 void loop() {
12     digitalWrite(TRIG_PIN, LOW);
13     delayMicroseconds(2);
14     digitalWrite(TRIG_PIN, HIGH);
15     delayMicroseconds(10);
16     digitalWrite(TRIG_PIN, LOW);
17
18     long duration = pulseIn(ECHO_PIN, HIGH);
19     float distance = duration * SOUND_SPEED / 2;
20
21     Serial.print("Distance: ");
22     Serial.print(distance);
23     Serial.println(" cm");
24     delay(500);
25 }
```

- **Simulation sur Tinkercad**

- Crée un nouveau circuit → ajoute un Arduino UNO + HC-SR04.
- Connecte les broches selon le tableau.
- Ouvre le Moniteur Série pour visualiser les distances (en cm).
- Déplace l'objet "Obstacle" dans la scène 3D pour voir la variation.



III. Exercice 2: Capteur de Lumière (LDR)

La **photo-résistance (LDR)** varie selon la lumière :



- Plus de lumière → résistance diminue.
- Moins de lumière → résistance augmente.

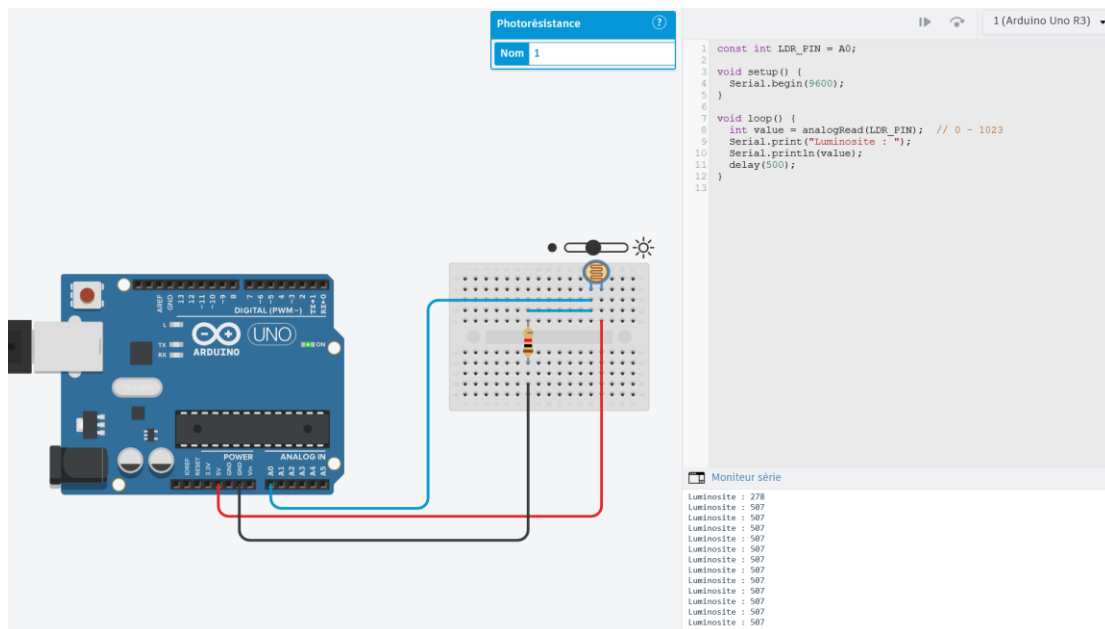
En la plaçant dans un **pont diviseur**, on mesure une tension proportionnelle à la luminosité.

- **Câblage**
 - LDR entre **5V** et **entrée analogique A0**
 - **Résistance 10 kΩ** entre **A0** et **GND**
- **Code Arduino**

```
1  const int LDR_PIN = A0;
2
3  void setup() {
4      Serial.begin(9600);
5  }
6
7  void loop() {
8      int value = analogRead(LDR_PIN); // 0 - 1023
9      Serial.print("Luminosité : ");
10     Serial.println(value);
11     delay(500);
12 }
```

- **Simulation sur Tinkercad**

- Ajoute une **LDR** et une **résistance** selon le schéma.
- Dans la simulation, clique sur la LDR pour **faire varier la luminosité**.
- Observe la valeur analogique dans le Moniteur Série.



IV. Exercice 3: Détection de mouvement avec un capteur PIR



Le **capteur PIR (Passive InfraRed)** détecte les **variations de rayonnement infrarouge** provoquées par le mouvement d'un corps chaud (comme un humain).

Il possède :

- Une **lentille Fresnel** qui concentre les rayonnements,
- Deux capteurs internes différentiels,
- Une **sortie numérique** :
 - **HIGH (1)** → mouvement détecté,
 - **LOW (0)** → aucune détection.

Il détecte les **changements** de chaleur, pas la présence constante.

- **Câblage sur Arduino UNO**

Broche capteur PIR	Arduino UNO
VCC	5V
GND	GND
OUT	D4

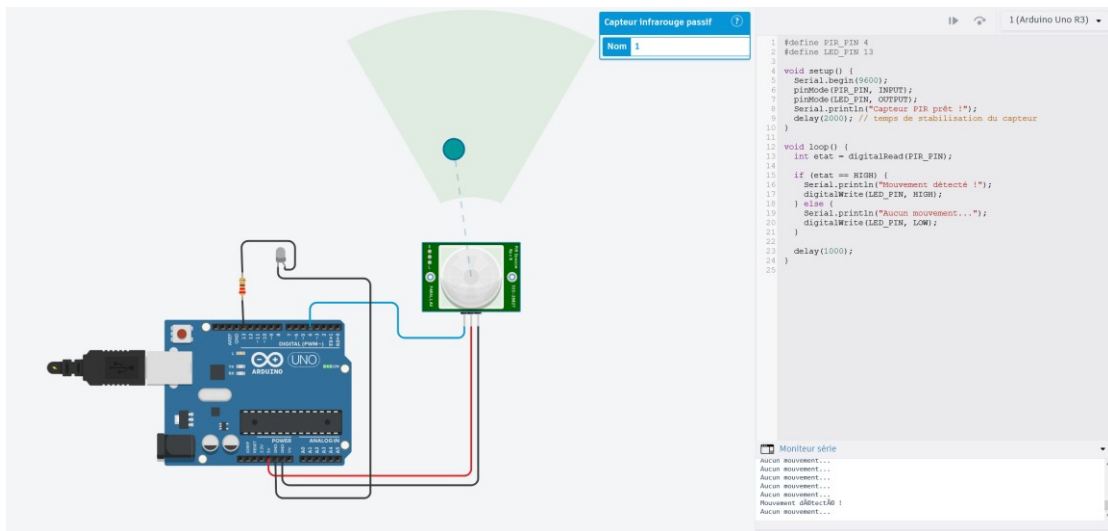
Ajoute une **LED** sur **D13** avec résistance 220 Ω pour visualiser la détection.

- **Code Arduino**

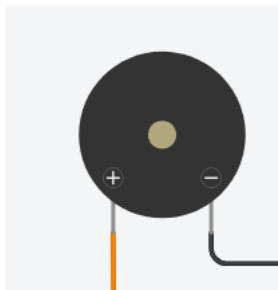
```
1  #define PIR_PIN 4
2  #define LED_PIN 13
3  void setup() {
4      Serial.begin(9600);
5      pinMode(PIR_PIN, INPUT);
6      pinMode(LED_PIN, OUTPUT);
7      Serial.println("Capteur PIR prêt !");
8      delay(2000); // temps de stabilisation du capteur
9  }
10 void loop() {
11     int etat = digitalRead(PIR_PIN);
12     if (etat == HIGH) {
13         Serial.println("Mouvement détecté !");
14         digitalWrite(LED_PIN, HIGH);
15     } else {
16         Serial.println("Aucun mouvement...");
17         digitalWrite(LED_PIN, LOW);
18     }
19     delay(1000);
20 }
```

- **Simulation sur Tinkercad**

- Crée un nouveau circuit.
- Ajoute :
 - 1 Arduino Uno
 - 1 capteur PIR
 - 1 LED
- Connecte les broches selon le tableau ci-dessus.
- **Clique sur le capteur PIR** → tu peux simuler la détection en cochant “Mouvement détecté”.
- Observe le message dans le Moniteur Série et la LED qui s’allume.



- **Ajouter un buzzer qui sonne lors d’un mouvement avec le code Arduino suivant :**



```
#define BUZZER_PIN 8
...
pinMode(BUZZER_PIN, OUTPUT);
...
if (etat == HIGH) {
    tone(BUZZER_PIN, 1000, 200); // émet
    un son bref
}
```

Explication de la syntaxe générale de **Tone** :

tone(pin, frequency, duration);

Paramètre	Signification	Exemple
pin	Numéro de la broche de sortie sur laquelle est connecté le buzzer	BUZZER_PIN (ex. 8)
frequency	Fréquence du son en Hertz (Hz)	1000 Hz = 1 kHz
duration	Durée du son en millisecondes (ms)	200 ms = 0,2 s